

Capa de Transporte

Introducción

Es la capa intermedia entre la capa de red y la de aplicación. Esta se encarga de llevar los mensajes desde la aplicación hasta la capa de red. No se involucra en como estos datos se transmiten en esta capa. Funcionan dentro de la terminal.

IP (el protocolo usado en la capa de red) utiliza el modelo de entrega de mejor esfuerzo, por lo que hace todo lo posible por entregar los paquetes, pero sin asegurar la llegada de los mismos. Por lo que se dice que es no fiable. TCP y UDP se encargan de ampliar el servicio de entrega de información.

Extender la entrega de host a host, a una entrega proceso a proceso, es lo que se llama multiplexación y desmultiplexación.

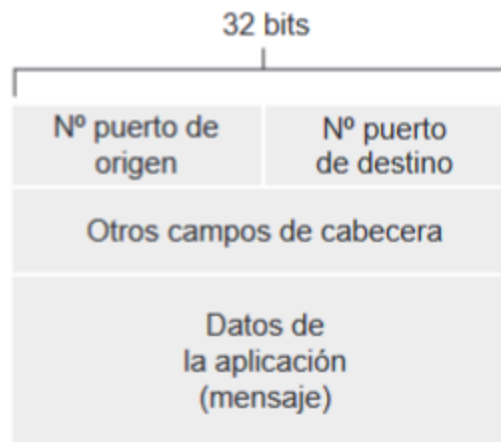
Multiplexación y desmultiplexación.

Al tener las aplicaciones de red uno o más sockets, la capa de transporte no entrega los paquetes directamente de la capa de red al proceso, sino que lo entrega a cada socket. En el extremo receptor, cada segmento va a tener un conjunto de campos destinados a dirigir el segmento con su respectivo socket. La capa de transporte examina estos campos, identifica al socket receptor y envía el segmento a dicho socket. Esto se llama desmultiplexación.

La tarea de reunir los fragmentos de datos en el host de origen desde los diferentes sockets, encapsulando cada fragmento de datos con la info de cabecera y pasarlos a la capa de red es lo que se denomina multiplexación.

Para ilustrar la tarea de desmultiplexación, recordemos la analogía doméstica de la sección anterior. Cada uno de los niños está identificado por su nombre. Cuando Bill recibe un lote de cartas del servicio de correos, lleva a cabo una operación de desmultiplexación al determinar a quién van dirigidas las cartas y luego las entrega en mano a sus hermanos. Ann lleva a cabo una operación de multiplexación al recopilar las cartas de sus hermanos y entregar todas las cartas al cartero

Los segmentos deben tener identificadores. Estos son el **número de puerto origen** y el **número de puerto destino**. La forma entonces de un segmento es la siguiente:



▼ UDP

Suponga que un proceso del host A, con el puerto UDP 19157, desea enviar un fragmento de datos de una aplicación a un proceso con el puerto UDP 46428 en el host B. La capa de transporte del host A crea un segmento de la capa de transporte que incluye los datos de aplicación, el número de puerto de origen (19157), el número de puerto de destino (46428) y otros dos valores (que veremos más adelante, pero que por el momento no son importantes para el tema que nos ocupa). La capa de transporte pasa a continuación el segmento resultante a la capa de red. La capa de red encapsula el segmento en un datagrama IP y hace el máximo esfuerzo por entregar el segmento al host receptor. Si el segmento llega al host receptor B, la capa de transporte del mismo examina el número de puerto de destino especificado en el segmento (46428) y entrega el segmento a su socket identificado por el puerto 46428. Observe que el host B podría estar ejecutando varios procesos, cada uno de ellos con su propio socket UDP y número de puerto asociado. A medida que los segmentos UDP llegan de la red, el host B dirige (demultiplexa) cada segmento al socket apropiado examinando el número de puerto de destino del segmento.

Es importante observar que un socket UDP queda completamente identificado por una tupla que consta de una dirección IP de destino y un número de puerto de destino. En consecuencia, si dos segmentos UDP tienen diferentes direcciones IP y/o números de puerto de origen, pero la misma dirección IP de destino y el mismo número puerto de destino, entonces los dos segmentos se enviarán al mismo proceso de destino a través del mismo socket de destino.

El número de puerto de destino en la cabecera sirve en caso de querer enviar una respuesta al host que envió el segmento.

▼ TCP

Una sutil diferencia entre un socket TCP y un socket UDP es que el primero queda identificado por una tupla de cuatro elementos: dirección IP de origen, número de puerto de origen, dirección IP de destino, número de puerto de destino. Por tanto, cuando un segmento TCP llega a un host procedente de la red, el host emplea los cuatro valores para dirigir (demultiplexar) el segmento al socket apropiado. En particular, y al contrario de lo que ocurre con UDP, dos segmentos TCP entrantes con direcciones IP de origen o números de puerto de origen diferentes (con la excepción de un segmento TCP que transporte la solicitud original de establecimiento de conexión) serán dirigidos a dos sockets distintos

El host servidor puede dar soporte a muchos sockets TCP simultáneos, estando cada socket asociado a un proceso y con cada socket identificado por su tupla de cuatro elementos. Cuando un segmento TCP llega al host, los cuatro campos (dirección IP de origen, puerto de origen, dirección IP de destino y puerto de destino) se utilizan para dirigir (demultiplexar) el segmento al socket apropiado.

UDP

▼ ¿Cómo funciona UDP?

UDP es lo más simple y sencillo que le puedes pedir a un protocolo de capa de transporte. Su trabajo es tomar los mensajes de la aplicación y pasárselos a la capa de red.

Incorpora lo justo y necesario que necesita un protocolo de capa de transporte, esto quiere decir que tiene multiplexación y desmultiplexación y un checksum para el manejo de errores. Aparte de eso, no añade nada a IP.

Se dice que UDP es un protocolo sin conexión, esto se debe a que no establece una conexión entre los dos hosts que se quieren comunicar a través de la red, sino que simplemente envía sus segmentos a la capa de red y si llegan bien y si no no pasa nada.

▼ Ventajas y desventajas

Probablemente, te preguntes por qué usaríamos UDP en lugar de TCP, bueno, yo te explico. Como desventaja sabemos que:

- No garantiza una entrega fiable de los datos entre un host y otro.
- No tiene un mecanismo de control de congestión, por lo que no se da cuenta si la red está sobrecargada o si él mismo está sobrecargando la red.

Sin embargo, las aplicaciones están mejor adaptadas a trabajar sobre UDP, ya que el mismo tiene:

- **Mejor control de la capa de aplicación sobre qué datos envían y cuando:** UDP los manda directamente, mientras que TCP regula la congestión y no va a mandar paquetes hasta que este todo ok. UDP sirve para aplicaciones que necesitan una velocidad mínima de transmisión o permiten la pérdida de algunos paquetes.
- **Sin establecimiento de la conexión:** UDP envía los datos sin ningún establecimiento previo de la conexión entre 2 hosts, por lo que no hay un retardo en la transferencia a la hora de enviar datos.
- **Sin información de la conexión:** TCP guarda un montón de info en los sistemas, terminales sobre el estado de la conexión, la congestión, los buffers, etc. UDP no guarda nada de esta info (aunque sea útil para la transferencia fiable de datos con TCP), lo cual permite que si se está usando en un servidor para una app concreta, pueda soportar la conexión de más hosts activos.
- **Poca sobrecarga por el tamaño de cabecera de los paquetes:** La cabecera de UDP pesa 8 bytes, mientras que la de TCP 20.

Aplicación	Protocolo de la capa de aplicación	Protocolo de transporte subyacente
Correo electrónico	SMTP	TCP
Acceso a terminales remotos	Telnet	TCP
Web	HTTP	TCP
Transferencia de archivos	FTP	TCP
Servidor de archivos remoto	NFS	Normalmente UDP
Flujos multimedia	Normalmente propietario	UDP o TCP
Telefonía por Internet	Normalmente propietario	UDP o TCP
Administración de red	SNMP	Normalmente UDP
Traducción de nombres	DNS	Normalmente UDP

Figura 3.6 ♦ Aplicaciones de Internet populares y sus protocolos de transporte subyacentes.

▼ Estructura de sus segmentos.

Su cabecera tiene 4 campos, cada uno ocupando 2 bytes. Tiene las 2 ips de los host, la suma de comprobación y la longitud del segmento.

▼ Checksum para detección de errores

Como funciona este checksum? Bueno suma todas las palabras de 16 bytes del campo de datos del segmento, calcula el complemento a 1 y te queda como checksum lo que te falta para que sea todos 1 en binario. Ahora bien, cuando un segmento llega a un host, realiza la suma y si no te queda todos 1 es porque alguno de los paquetes fue alterado durante el viaje.

Por que tiene control de errores? Simple, porque en la capa de red no todos los routers y switches tiene un mecanismo para control de errores, asi que al no haber nada mas abajo que se fije si los paquetes sufren errores, depende de la capa de transporte regular este tipo de fallos **terminal a terminal**.

TCP

▼ ¿Cómo funciona TCP?

Es un protocolo de capa de transporte fiable y orientado a la conexión. Se dice que es orientado a la conexión porque antes de empezar a transferir datos, los dos

hosts establecen primero una conexión entre ellos para definir los parámetros de transferencia.

▼ Estructura de segmento TCP

- El campo Número de secuencia de 32 bits y el campo Número de reconocimiento también de 32 bits son utilizados por el emisor y el receptor de TCP para implementar un servicio de transferencia de datos fiable, como se explica más adelante.
- El campo Ventana de recepción de 16 bits se utiliza para el control de flujo. Veremos en breve que se emplea para indicar el número de bytes que un receptor está dispuesto a aceptar.
- El campo Longitud de cabecera de 4 bits especifica la longitud de la cabecera TCP en palabras de 32 bits. La cabecera TCP puede tener una longitud variable a causa del campo opciones de TCP (normalmente, este campo está vacío, por lo que la longitud de una cabecera TCP típica es de 20 bytes).
- El campo Opciones es opcional y de longitud variable. Se utiliza cuando un emisor y un receptor negocian el tamaño máximo de segmento (MSS) o como un factor de escala de la ventana en las redes de alta velocidad. También se define una opción de marca temporal. Consulte los documentos RFC 854 y RFC 1323 para conocer detalles adicionales.
- El campo Indicador tiene 6 bits. El bit ACK se utiliza para indicar que el valor transportado en el campo de reconocimiento es válido; es decir, el segmento contiene un reconocimiento para un segmento que ha sido recibido correctamente. Los bits RST, SYN y FIN se utilizan para el establecimiento y cierre de conexiones, como veremos al final de esta sección. Los bits CWR y ECE se emplean en la notificación de congestión explícita, como veremos en la Sección 3.7.2. La activación del bit PSH indica que el receptor deberá pasar los datos a la capa superior de forma inmediata. Por último, el bit URG se utiliza para indicar que hay datos en este segmento que la entidad de la capa superior del lado emisor ha marcado como “urgentes”. La posición de este último byte de estos datos urgentes se indica mediante el campo puntero de datos urgentes de 16 bits. TCP tiene que informar a la entidad de la capa superior del lado receptor si existen datos urgentes y pasarle un puntero a la posición donde finalizan los datos urgentes. En la práctica, PSH, URG y el puntero a

datos urgentes no se utilizan. Sin embargo, hemos hablado de estos campos con el fin de proporcionar al lector la información completa.

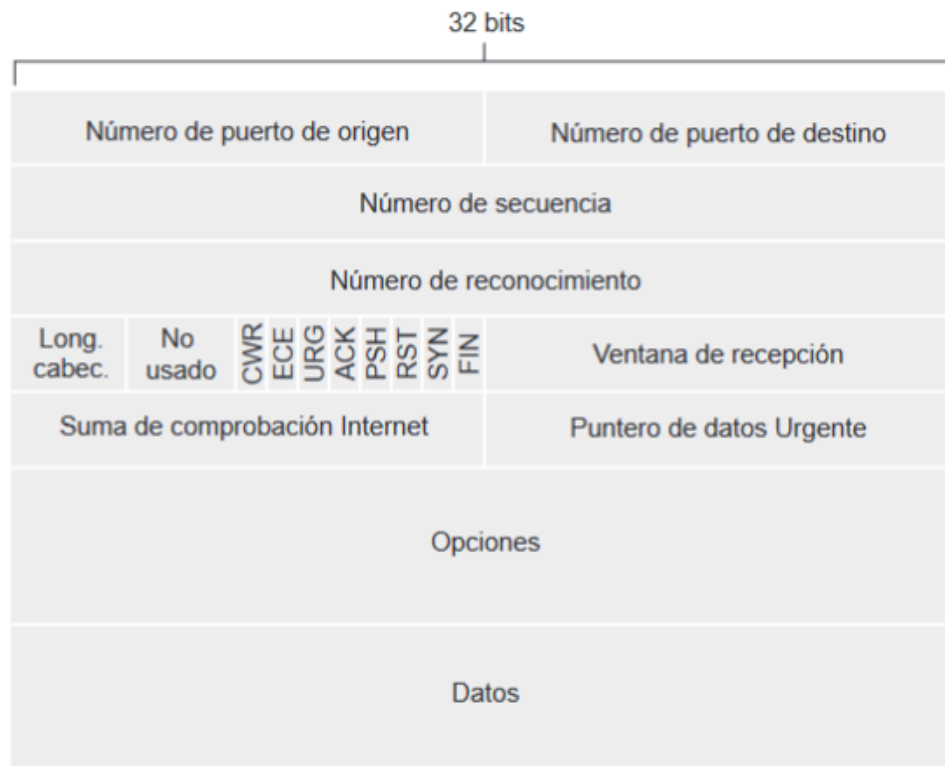


Figura 3.29 ♦ Estructura del segmento TCP.

▼ Ventajas y desventajas

Desventajas:

- La transferencia de datos puede ser lenta debido al control de congestión de la red y a la necesidad de transferir todos los paquetes.
- Se puede congestionar fácilmente la red debido al tamaño de cada uno de los segmentos TCP

Ventajas:

- Es orientado a la conexión, lo que quiere decir que ambos host terminales tienen registro de la estado de la conexión y la transferencia de datos durante la duración de la misma.

- Posee control de congestión, permitiendo enviar datos acorde a la velocidad de transferencia y a la congestión en la red.
- Es fiable, lo que quiere decir que puede retransmitir paquetes perdidos las veces que sean necesarias de forma que lleguen todos los paquetes y en orden.

▼ Transferencia de datos fiable

TCP usa un servicio de transferencia de datos fiable sobre un protocolo no fiable de la capa de red (si yo no lo hago, entonces nadie lo hará).

El servicio de transferencia de datos fiable de TCP garantiza que el flujo de datos que un proceso extrae de su buffer de recepción TCP no está corrompido, no contiene huecos, ni duplicados y está en orden; es decir, el flujo de bytes es exactamente el mismo flujo que fue enviado por el sistema terminal existente en el otro extremo de la conexión.

Para los segmentos que llegan y aún no se reconocen, TCP incorpora un temporizador de retransmisión para saber en base a la velocidad de transferencia previa cuando debería preocuparse por un segmento que no haya llegado (ya sea de confirmación o de envío).

Este temporizador es utilizado principalmente para la retransmisión en caso de que un segmento se haya perdido.

Esto funciona con 3 sucesos:

1. TCP recibe los datos, los guarda en un segmento y pasa el segmento a IP, cada segmento tiene un número de secuencia que es el número de datos del primer byte de datos del segmento.
2. Fin de la temporización, TCP responde a esto retransmitiendo el segmento que causó el fin de la temporización y reinicia el temporizador
3. La llegada de un segmento de reconocimiento (ACK). Cuando llega lo compara con su número de secuencia más antiguo. La forma de validar de TCP es acumulativa, lo que quiere decir que si comprueba que llegaron todos los segmentos anteriores y el nuevo es mayor al número más antiguo, entonces el ACK confirma uno o más segmentos anteriores.

Ahora, el periodo de retransmisión puede ser relativamente largo, así que se incorpora algo llamado retransmisión rápida. Esto funciona detectando ack duplicados. Un ack duplicado sucede cuando un host recibe un segmento cuyo número de secuencia es mayor al número que estaba esperando. (Esperaba el 2 y le llega el 3), TCP lo que hace al no usar NACK para decir que le falta un segmento es confirmar nuevamente el último segmento que recibió anterior al hueco (por ejemplo el 1).

Dado que se mandan muchos segmentos seguidos, van a haber muchos duplicados. Cuando el emisor TCP detecta 3 acks duplicados, lo toma como que el segmento que le sigue al duplicado se perdió, entonces realiza una retransmisión rápida del segmento que falta antes de que se cumpla el fin de temporización de ese segmento.

TCP tiene también algo que se llama reconocimiento selectivo, es decir, que guarda los segmentos que le llegan (no necesariamente en orden) en un buffer, y los va reconociendo en orden, es decir que si se pierde un segmento, pero llegan los 3 siguientes, no solicita todos sino solo el que se perdió y agarra el resto de su buffer de recepción.

▼ Control de flujo

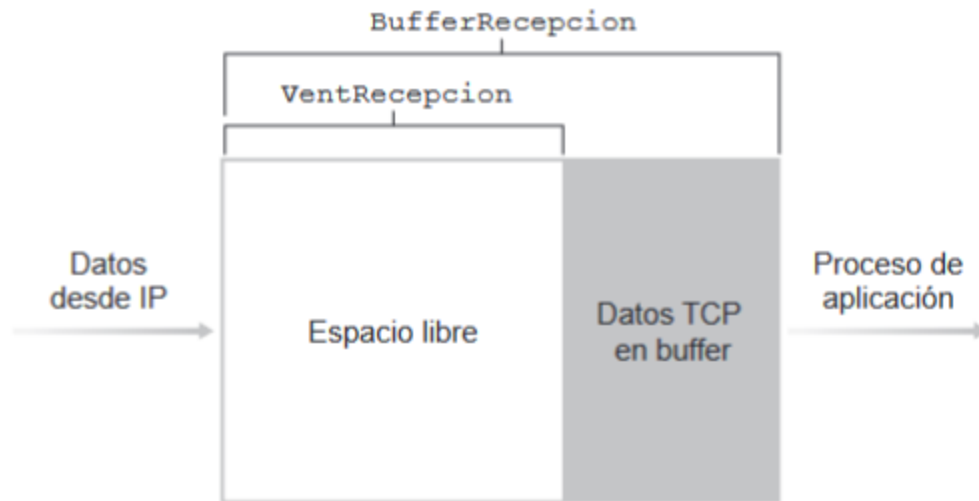
TCP proporciona un servicio de control de flujo a sus aplicaciones para eliminar la posibilidad de que el emisor desborde el buffer del receptor. El control de flujo es, por tanto, un servicio de adaptación de velocidades (adapta la velocidad a la que el emisor está transmitiendo frente a la velocidad a la que la aplicación receptora está leyendo).

TCP proporciona un servicio de control de flujo teniendo que mantener el emisor una variable conocida como ventana de recepción. Informalmente, la ventana de recepción se emplea para proporcionar al emisor una idea de cuánto espacio libre hay en el buffer del receptor. Puesto que TCP es una conexión full-dúplex, el emisor de cada lado de la conexión mantiene una ventana de recepción diferente

En resumen, el control de flujo resuelve el problema de la capacidad de datos que es capaz de recibir el cliente, minimizando lo más posible la pérdida de paquetes, debido a no ser capaz de recibirlos en su buffer

Para esto, TCP utiliza un mecanismo llamado “ventana deslizante” para realizar los controles de flujo de datos, esto se utiliza para controlar la cantidad de información

que un dispositivo recibe, regulando la entrada de los mismos



El cliente le indica al servidor el tamaño de su ventana de recepción al iniciar la conexión, haciendo que el servidor no pueda enviar más datos que el tamaño de dicha ventana, teniendo que esperar la confirmación de recepción de los mismos para poder enviar nuevos datos

▼ Gestión de conexión

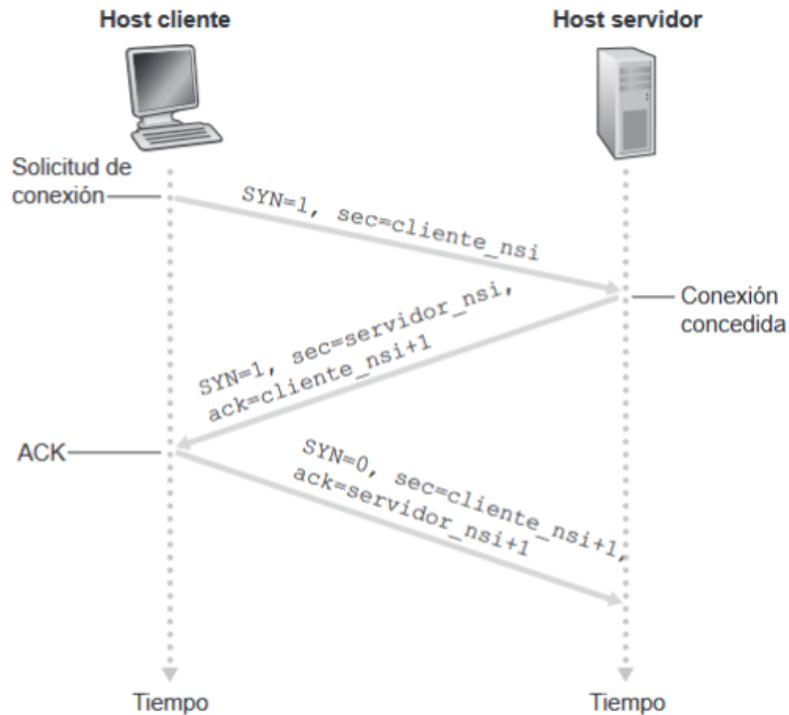


Figura 3.39 ♦ El proceso de acuerdo en tres fases de TCP: intercambio de segmentos.

Esta conexión previa se denomina 3-way handshake, y consiste básicamente en que

- El cliente (el que inicia la conexión) envía un mensaje SYN al servidor (el que recibe la conexión).
- Posteriormente, el servidor le enviará un mensaje de tipo SYN-ACK, indicando que puede empezar a enviar información,
- Finalmente, el cliente envía un ACK indicando que lo ha recibido correctamente, y ya se empieza a enviar toda la información entre cliente y servidor de manera bidireccional.

Suceso	Acción del receptor TCP
Llegada de un segmento en orden con el número de secuencia esperado. Todos los datos hasta el número de secuencia esperado ya han sido reconocidos.	ACK retardado. Esperar hasta durante 500 milisegundos la llegada de otro segmento en orden. Si el siguiente segmento en orden no llega en este intervalo, enviar un ACK.
Llegada de un segmento en orden con el número de secuencia esperado. Hay otro segmento en orden esperando la transmisión de un ACK.	Enviar inmediatamente un único ACK acumulativo, reconociendo ambos segmentos ordenados.
Llegada de un segmento desordenado con un número de secuencia más alto que el esperado. Se detecta un hueco.	Enviar inmediatamente un ACK duplicado, indicando el número de secuencia del siguiente byte esperado (que es el límite inferior del hueco).
Llegada de un segmento que completa parcial o completamente el hueco existente en los datos recibidos.	Enviar inmediatamente un ACK, suponiendo que el segmento comienza en el límite inferior del hueco.

Tabla 3.2 ♦ Recomendación para la generación de mensajes ACK en TCP [RFC 5681].

Todo lo bueno se termina y esto es aplicable también a una conexión TCP. Cualquiera de los dos procesos participantes en una conexión TCP pueden dar por terminada dicha conexión.

El proceso de la aplicación cliente ejecuta un comando de cierre. Esto hace que el cliente TCP envíe un segmento especial TCP al proceso servidor. Este segmento especial contiene un bit indicador en la cabecera del segmento, el bit FIN puesto a 1. Cuando el servidor recibe este segmento, devuelve al cliente un segmento de reconocimiento. El servidor entonces envía su propio segmento de desconexión, que tiene el bit FIN puesto a 1. Por último, el cliente reconoce el segmento de desconexión del servidor. En esta situación, los recursos de ambos hosts quedan liberados.

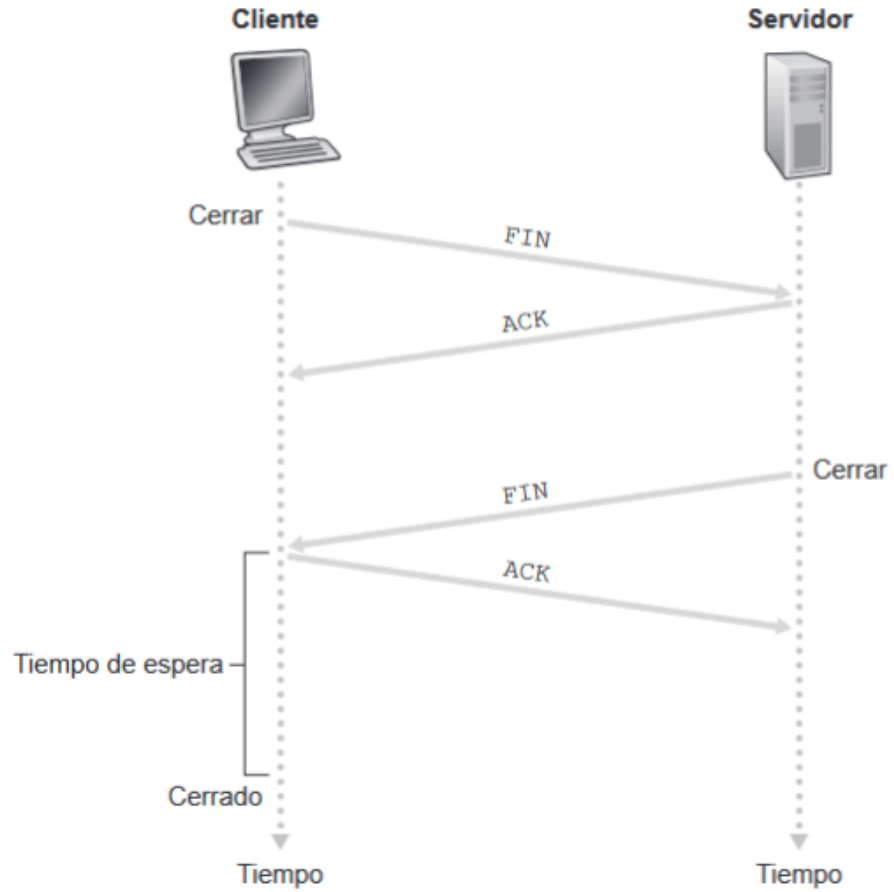


Figura 3.40 ♦ Cierre de una conexión TCP.

<https://www.youtube.com/watch?v=zyoxBGRdFmg>

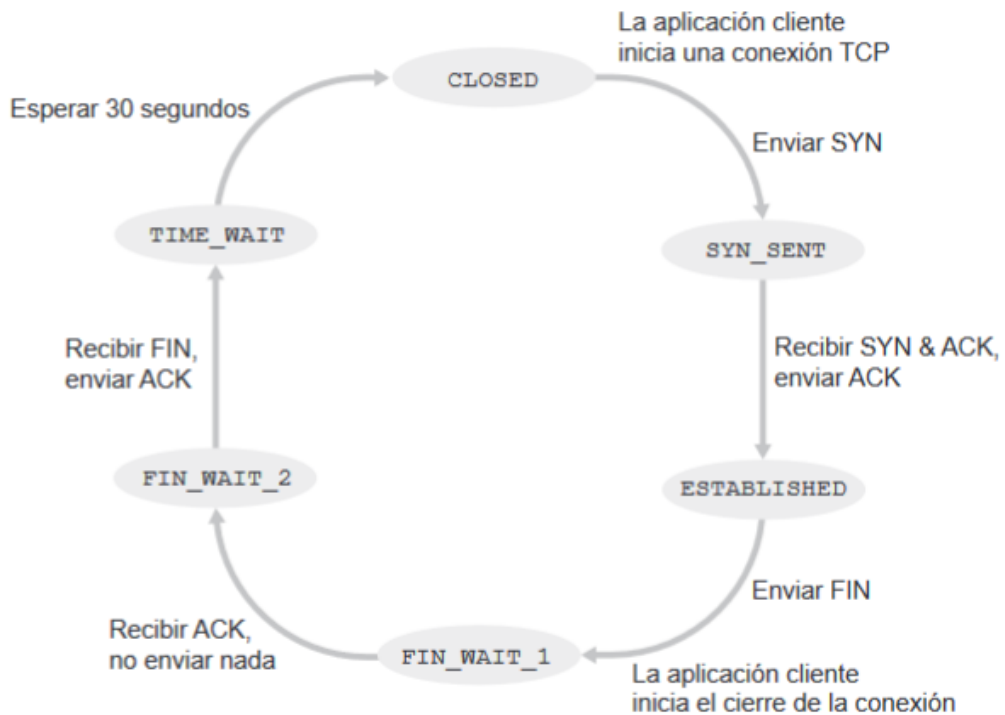


Figura 3.41 ♦ Secuencia típica de estados TCP visitados por un cliente TCP.

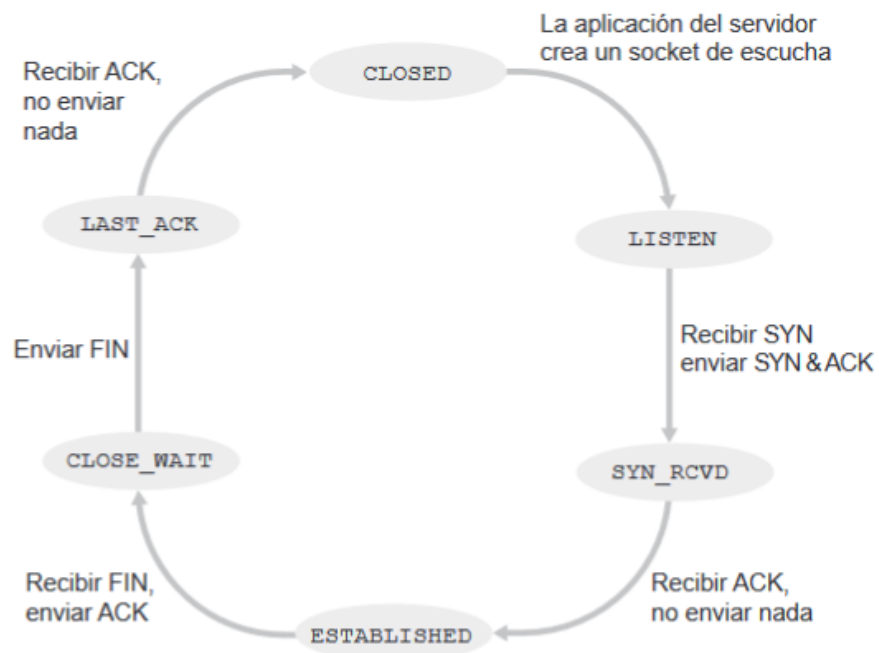


Figura 3.42 ♦ Secuencia típica de estados TCP visitados por un servidor TCP.

▼ Control de congestión

Para controlar la congestión de datos, TCP utiliza una ventana de congestión, con la cual puede determinar si es necesario modificar la velocidad de transferencia de paquetes

La ventana de congestión parte de una idea básica, si se pierden paquetes, se asume que hay congestión en la red y dicha pérdida se evalúa en función de los paquetes de reconocimiento recibidos

De esta forma, si se determina que existe congestión, modifica la ventana de forma que el servidor ralentice el envío de paquetes. En caso contrario, si determina que no hay congestión, se modificará la ventana, de forma que el servidor pueda enviar más paquetes

- Arranque lento (Slow Start)
 1. El algoritmo slow-start plantea iniciar con una velocidad lenta de transmisión setando el MSS en un tamaño pequeño de transferencia de paquetes e ir incrementándolo exponencialmente a medida que los ACK de respuesta van llegando, a su vez la ventana de congestión aumenta exponencialmente junto con el MSS, hasta que el servidor no recibe el ACK de confirmación de recepción de paquete, en cuyo caso se toma como una señal de congestión en la red.
 2. La ventana de congestión se setea a 1 y se establece una variable umbralAL que toma de valor la mitad del valor del tamaño de la ventana de congestión cuando se produce la pérdida de paquete. Luego se vuelve a incrementar exponencialmente la ventana de congestión hasta alcanzar el valor del umbralAL, donde se pasa a estado de congestión avoidance

- Evitación de la congestión (Congestion-Avoidance)

Una vez alcanzado el umbral, se procede a incrementar la ventana de congestión en 1 MSS (de forma lineal) por cada RTT, hasta que se detecte nuevamente congestión, pero por recibir los 3 ack duplicados. En cuyo caso se pasa al estado de fast recovery

- Recuperación rápida

En la fase de recuperación rápida, el valor de VentCongestion se incrementa en 1 MSS por cada ACK duplicado recibido correspondiente al segmento que

falta y que ha causado que TCP entre en el estado de recuperación rápida. Cuando llega un ACK para el segmento que falta, TCP entra de nuevo en el estado de evitación de la congestión después de disminuir el valor de VentCongestion. Si se produce un fin de temporización, el mecanismo de recuperación rápida efectúa una transición al estado de arranque lento después de realizar las mismas acciones que en los modos de arranque lento y de evitación de la congestión: el valor de VentCongestion se establece en 1 MSS y el valor de umbralAL se hace igual a la mitad del valor que tenía VentCongestion cuando tuvo lugar el suceso de pérdida.

